

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Yazılım Geliştirme I

Kaynak Yönetimi ve Akıllı İşaretçiler

Hafta 4

Kaynak Yönetimi ve Akıllı İşaretçiler

Doç. Dr. Tolga Berber

Bilgisayar Bilimleri · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Hafta 2'de `araclar::IntTampon`'u kurdunuz — kurucusu `new[]` ile bellek ayırıyor, yıkıcısı `delete[]` ile bırakıyordu; ama onu hiç kopyalamamıştınız.

- Şimdi masum görünen tek bir satır yazın:

```
IntTampon b = a;
```

- Derleyici hiç uyarmadan iki nesneyi aynı belleğe bağlar; program bittiğinde aynı blok iki kez silinmeye çalışılır
- Kopyalama, üzerinde hiç durmadığınız bu sessiz tuzağı taşır — bu hafta onu güvenle ele alacaksınız

Bu Hafta

◎ Öğrenme Hedefleri

Bu hafta sonunda şunları yapabileceksiniz:

1. Varsayılan (sığ) kopyanın kaynak tutan sınıflarda neden tehlikeli olduğunu açıklamak
2. Doğru bir kopya kurucu ve kopya atama operatörü (derin kopya) yazmak
3. Rule of 3'ü, `=default`/`=delete` sözdizimini uygulamak
4. `unique_ptr`/`shared_ptr`'ı `make_unique`/`make_shared` ile kullanmak
5. Akıllı işaretçiyle yönetilen basit bir bağlı liste (`IntBagliListe`) tasarlamak

BÖLÜM 1

Ünite 1 — Kopyalama Sorunu

2 • İçerik Hatırlatma: `araclar::IntTampon`

Hafta 2'de tanıdığınız `IntTampon`, kurucusu `new[]` ile ayırıp yıkıcısı `delete[]` ile bıraktığı bir `int*` tutar:

```
8   int* veri;
9   int boyut;
```

- Kurucu/yıkıcı Hafta 3'te biçimsel terimlerle adlandırıldı — ama kopyalama hiç ele alınmadı
- `IntTampon b = a;` yazılırsa ne olur? Bugün cevaplıyoruz.

Varsayılan Kopya: Sığ ve Tehlikeli

Siz hiçbir şey yazmazsanız derleyici bir varsayılan kopya kurucu üretir; bu, üyeleri üye üye (memberwise) kopyalar:

- Bir işaretçi üyesi için bu, işaretçinin *kendisini* (adresi) kopyalamak demektir — işaret ettiği veriyi DEĞİL
- `IntTampon b = a;` sonrası `a.veri` ve `b.veri` aynı heap adresini gösterir (sığ kopya)
- Sonuç: biri değiştirirse öteki de değişir; ikisi de kapsam dışına çıkınca aynı bellek iki kez `delete[]` edilir (çifte `delete`)

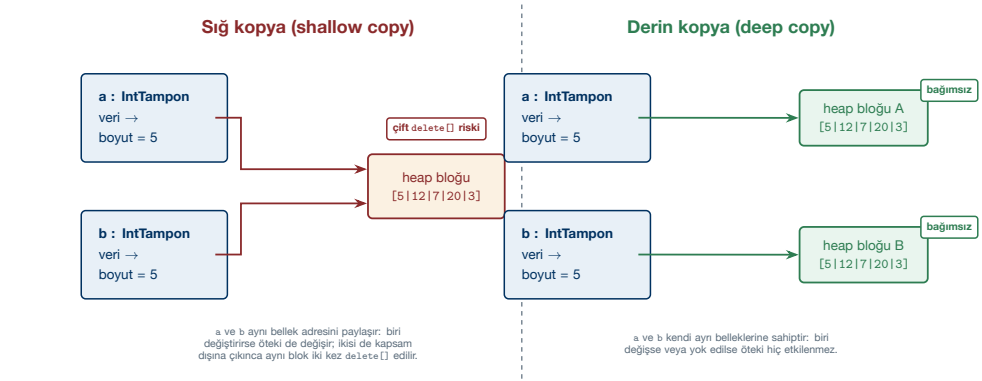
Analoji: Aynı Eve İki Zarf

Analoji

İki zarfa da aynı ev adresini yazıp gönderirsiniz: ikisi de aynı eve açılır. Zarflardan biri “evi yık” derse öbür zarfın sahibi de evsiz kalır; ikisi de “evi yık” derse aynı ev iki kez yıkılmaya çalışılır. Oysa evin birebir kopyasını yeni bir arsada inşa etseydiniz, artık iki bağımsız eviniz olurdu — biri yıkılsa öteki hiç etkilenmezdi.

- Aynı adresi paylaşan iki zarf $\rightarrow$ sığ kopya
- Yeni arsada inşa edilen birebir kopya $\rightarrow$ derin kopya

Sığ Kopya vs Derin Kopya



Solda: iki nesne aynı bloğu paylaşır (çift `delete` riski). Sağda: her nesnenin kendi bağımsız bloğu vardır.

Çözüm: Kopya Kurucu

```
23 IntTampon::IntTampon(const IntTampon& diger) {
24    boyut = diger.boyut;
25    veri = new int[boyut];
26    for (int i = 0; i < boyut; ++i) {
27       veri[i] = diger.veri[i];
28    }
29    std::cout << "tampon kopyalandi (derin kopya, boyut=" << boyut <<
          ")\n";
30 }
```

- Kendi `new[]`'ini yapar, veriyi elemanlarıyla kopyalar (derin kopya) — artık bağımsız iki kaynak vardır
- Bildirimi `IntTampon(const IntTampon& diger);`

Çözüm: Kopya Atama Operatörü

```
34 IntTampon& IntTampon::operator=(const IntTampon& diger) {
35    if (this == &diger) {
36       return *this; // oz-atama kontrolu
37    }
38    delete[] veri; // eski kaynak birakilir
39    boyut = diger.boyut;
40    veri = new int[boyut];
41    for (int i = 0; i < boyut; ++i) {
42       veri[i] = diger.veri[i];
43    }
44    std::cout << "tampon atandi (derin kopya, boyut=" << boyut << ")\n
          ";
45    return *this;
46 }
```

BÖLÜM 2

Ünite 2 — Rule of 3

2 • İçerik Rule of 3

Tanım: Rule of 3

Bir sınıf yıkıcı, kopya kurucu veya kopya atama operatöründen *birini* elle yazmak zorundaysa genelde *üçünü de* yazmalıdır — üçü de aynı soruyu (“kaynağın sahibi bu sınıf mı?”) cevaplar.

Analoji

Bir kaynağı özel yönetmeye karar verdiğinizde (garson yerine kendiniz servis yapıyorsunuz gibi), set menünün üç parçasını birlikte sipariş etmeniz gerekir — yalnız birini alıp diğerlerini “standart” bırakmak tutarsızlığa yol açar.

=default / =delete

```
7  class Sayac {
8  public:
9     explicit Sayac(int baslangic = 0) : deger_(baslangic) {}
10    Sayac(const Sayac&) = default;
11    Sayac& operator=(const Sayac&) = default;
12    int deger() const { return deger_; }
13    void artir() { ++deger_; }
14
15 private:
16    int deger_;
17 };
```

- `=default`: `Sayac` kaynak tutmaz, memberwise kopya açıkça istenir
- `=delete`: `TekilKart` kopyalanamaz, deneme derleme hatası verir

Kısa Bakış: Taşıma (*move*) Semantiği

`std::move(x)` ne yapar?

x'i bir "rvalue referansı" olarak işaretler — taşıma kurucusu yoksa yine derlenir ama kopya kurucuya düşer (güvenli, fakat gereksiz derin kopya nedeniyle verimsiz).

```
38    std::cout << "-- tasima onizleme --\n";
39    araclar::IntTampon d(std::move(c));
40    // Not: IntTampon icin kullanici tanimli bir tasima kurucusu
          YAZILMADI;
41    // std::move(c) yine de derlenir ama burada -- tasima kurucusu
42    // olmadigi icin -- kopya kurucuya duser (guvenli, fakat gereksiz
          bir
43    // derin kopya yapilir; verimsiz). c bu yuzden hala gecerli kalir.
44    std::cout << "d.veri[1]=" << d.veri[1] << "\n";
```

BÖLÜM 3

Ünite 3 — Akıllı İşaretçiler

2 • İçerik `unique_ptr`: Tekil Sahiplik

Analoji

`unique_ptr` devredilebilir ama çoğaltılamaz tek bir bagaj fişine benzer: fiş elden ele verilebilir (taşınabilir, `std::move`) ama fotokopisi çekilemez (kopyalanamaz); fiş kimdeyse bagajın sahibi odur — aynı anda iki kişide aynı fiş olamaz.

- Kaynağı yalnızca bir `unique_ptr` aynı anda sahiplenir
- Kopya kurucu/atama örtük `=delete` — yalnız taşınabilir
- `std::make_unique<T>(...)` ile oluşturulur; çıplak `new` yazılmaz

AkilliTampon: Rule of 0 Uygulaması

```
8  class AkilliTampon {
9  public:
10    explicit AkilliTampon(int n);
11    int boyut() const { return boyut_; }
12    int& operator[](int i) { return veri_[i]; }
13    int operator[](int i) const { return veri_[i]; }
14
15 private:
16    std::unique_ptr<int[]> veri_;
17    int boyut_;
18 };
```

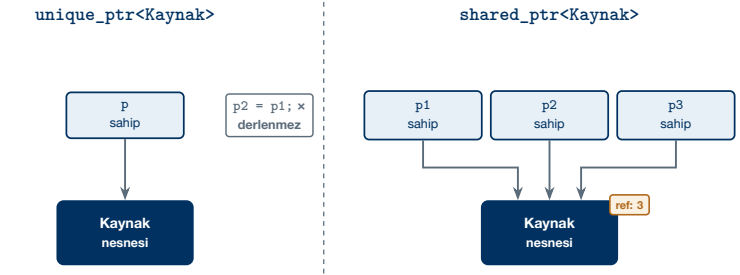
- `unique_ptr<int[]>` üyesi kopyalanamaz olduğu için sınıf kendiliğinden kopyalanamaz hale gelir — kullanıcı tanımlı yıkıcı/kopya yazılmadı

shared_ptr: Paylaşılan Sahiplik

Ortak bir aile aboneliği gibi: kaç kişi kullanıyorsa sayaç o kadardır, son kişi ayrılınca abonelik kapanır

```
27    std::cout << "-- shared_ptr demo --\n";
28    auto p1 = std::make_shared<araclar::IntTampon>(5);
29    std::cout << "p1.use_count()=" << p1.use_count() << "\n";
30    {
31       auto p2 = p1; // paylasilan sahiplik, sayac artar
32       std::cout << "p1.use_count()=" << p1.use_count()
33                 << " p2.use_count()=" << p2.use_count() << "\n";
34    } // p2 kapsam disina cikar, sayac azalir
35    std::cout << "p1.use_count()=" << p1.use_count() << "\n";
```

`unique_ptr` vs `shared_ptr`



Varsayılan tercih `unique_ptr`; birden çok sahip aynı anda gerekiyorsa `shared_ptr` (`weak_ptr` kapsam dışı).

BÖLÜM 4

Ünite 4 — `IntBagliListe`

2 • İçerik Düğüm: `Dugum`

Kütüphanenin ilk bağlı yapısı — tek yönlü, somut tipli (yalnız `int`) bağlı liste:

```
18  struct Dugum {
19      int deger;
20      std::unique_ptr<Dugum> sonraki;
21  };
```

- `sonraki`, kendi türünden bir sonraki düğümü sahiplenir
- Listenin tek üyesi `unique_ptr<Dugum> bas_` — yıkıcı elle yazılmaz, `bas_` yok edilince zincir otomatik yok edilir

ekleBasa: Sahiplik Zincirini Kaydırmak

```
8  void IntBagliListe::ekleBasa(int deger) {
9     auto yeniDugum = std::make_unique<Dugum>();
10    yeniDugum->deger = deger;
11    yeniDugum->sonraki = std::move(bas_); // eski bas, yeni dugumun
          sonrakisi olur
12    bas_ = std::move(yeniDugum); // sahiplik yeni dugume kayar
13 }
```

- Yeni düğüm `make_unique` ile kurulur
- Mevcut `bas_`, yeni düğümün `sonraki`'sine `std::move` ile aktarılır, sonra yeni düğüm `bas_`'a taşınır

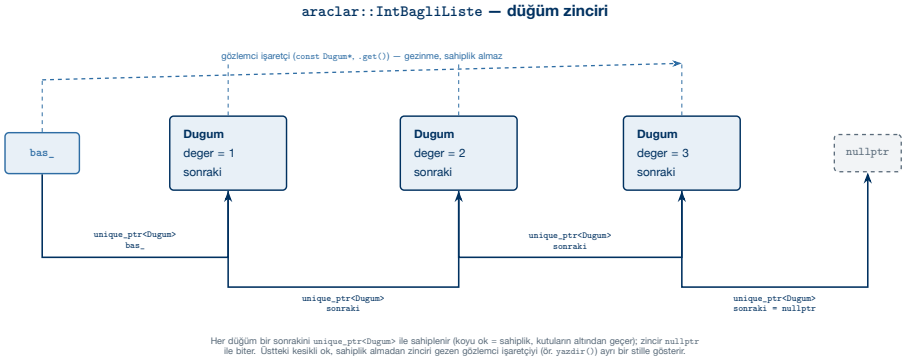
`yazdir`: Gözlemci Ham İşaretçi

Bir kitapçıda rafları gezen müşteri gibi: kitapları görür ama hiçbirinin sahibi değildir

```
17 void IntBagliListe::yazdir() const {
18    const Dugum* gozlemci = bas_.get(); // sahiplik almayan gozlemci
         isaretci
19    while (gozlemci != nullptr) {
20       std::cout << gozlemci->deger << " -> ";
21       gozlemci = gozlemci->sonraki.get();
22    }
23    std::cout << "nullptr\n";
24 }
```

Gezinme `.get()` ile yapılır — sahiplik gerekmez.

Düğüm Zinciri



`bas_`'tan başlayıp `sonraki` sahiplik oklarıyla ilerleyen zincir; son düğümden çıkan ok `nullptr`'a gider.

BÖLÜM 5

Özet

Özet

Özet

- Varsayılan kopya sığdır — kaynak tutan sınıfta derin kopya elle yazılır
- Rule of 3: biri elle yazılırsa üçü de yazılır; `=default`/`=delete` niyeti belirtir
- `unique_ptr`: tekil/taşınabilir; `shared_ptr`: paylaşılan/sayımlı sahiplik
- `AkilliTampon`: Rule of 0 — akıllı işaretçiyle hiçbir özel fonksiyon elle yazılmaz
- `IntBagliListe`: düğümlerini `unique_ptr` ile yöneten ilk bağlı yapı

5 • İzleme Aktarım — Başka Nerede?

“Bir kaynağı kim sahiplenir ve kopyalanırken ne olmalı?” sorusu yalnız `IntTampon`’a özgü değildir:

- Bellek, dosya tanıtıcısı veya ağ bağlantısı tutan her sınıf aynı soruyla karşılaşır: kaynağı paylaş mı, yoksa yeni bir kaynak mı ayır?
- Kitaplık projesinde ileride bir kaynak tutacak her tip için aynı derin/sığ kopya kararını verecek; paylaşım gerektiğinde `shared_ptr`, tek sahiplikte `unique_ptr` seçeceksiniz
- Sahiplik ile gezinmeyi ayırma alışkanlığı ileride her ağaç/graf gibi bağlı yapıda karşınıza çıkacak

Sırada Ne Var?

Bu hafta lab'da

`IntTampon`'a kopya kurucu/atama *kendi elinizle* yazacak, `AkilliTampon` ve `IntBagliListe`'yi akıllı işaretçilerle derleyip çalıştıracaksınız.

- Ders notundaki Kendinizi Sınayın soruları her üniteyi kapsıyor: sığ kopya neden tehlikeli, `TekilKart` neden `=delete`, `unique_ptr`/`shared_ptr` farkı, `yazdir()` neden ham işaretçi kullanır
- Gelecek hafta: bu nesneleri dilin yerleşik operatörleriyle (`[]`, `<<`, karşılaştırma) doğal biçimde kullanmak — Operatör Yükleme